

# Squeezing Every FLOP: Single-GPU Training Efficiency

## Optimization for Vision-Language Models

A Case Study on SiQ-VL with NVIDIA Blackwell

Duo An  
duo.an@example.com

May 2026

### Abstract

We present a systematic study of single-GPU training efficiency optimization for SiQ-VL, a vision-language model trained on NVIDIA RTX PRO 6000 Blackwell (96 GB VRAM). Through 15 iterations of measured optimizations—spanning dtype fixes, kernel fusion (Liger-Kernel, TileGym cuTile), data strategies (bucketing, packing), and batch scaling—we improve throughput from **4,674 to 107,367** real tokens/second: a **22.9× end-to-end speedup**. We document both successes and failures: vision feature caching (0% gain), Triton Flash Attention (slower than SDPA), FlexAttention packing (51% waste), and TileGym with variable shapes (5× regression). The final configuration achieves MFU $\approx$ 22.7% on the 0.5B model, confirmed to be near the memory-bandwidth ceiling for this model scale.

**Keywords:** Vision-Language Models, Training Efficiency, GPU Kernel Optimization, cuTile, Liger-Kernel, Blackwell, Sequence Packing

## Contents

|          |                                                                       |          |
|----------|-----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                   | <b>3</b> |
| 1.1      | Optimization Timeline . . . . .                                       | 3        |
| 1.2      | Contributions . . . . .                                               | 3        |
| <b>2</b> | <b>System Setup</b>                                                   | <b>4</b> |
| <b>3</b> | <b>The Optimization Journey</b>                                       | <b>4</b> |
| 3.1      | Iteration 0 — Baseline Measurement . . . . .                          | 4        |
| 3.2      | Iteration 1 — Fix FP32 + Vision <code>no_grad</code> . . . . .        | 5        |
| 3.3      | Iteration 2 — Liger-Kernel Integration . . . . .                      | 5        |
| 3.4      | Iteration 3 — Vision Acceleration Investigation . . . . .             | 6        |
| 3.5      | Iteration 5 — Length Bucketing . . . . .                              | 6        |
| 3.6      | Iteration 7 — <code>torch.compile</code> Replaces Liger . . . . .     | 7        |
| 3.7      | Iteration 8 — Batch Tuning Under <code>torch.compile</code> . . . . . | 7        |
| 3.8      | Iteration 10 — Sequence Packing + FlexAttention . . . . .             | 8        |
| 3.9      | Iteration 11 — TileGym FA4: Blackwell-Native Attention . . . . .      | 8        |
| 3.10     | Iteration 13 — TileGym Full Stack: ALL Qwen2 Ops . . . . .            | 9        |
| 3.11     | Iteration 15 — Ground Truth Verification . . . . .                    | 9        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>4</b> | <b>Results</b>                                           | <b>10</b> |
| 4.1      | The Full Speedup Progression . . . . .                   | 10        |
| 4.2      | Verified Ground-Truth Results (Iteration 15) . . . . .   | 11        |
| 4.3      | Padding Waste Analysis . . . . .                         | 11        |
| 4.4      | VRAM-Throughput Pareto Frontier . . . . .                | 12        |
| 4.5      | TileGym vs Liger-Kernel . . . . .                        | 12        |
| 4.6      | MFU Analysis . . . . .                                   | 13        |
| <b>5</b> | <b>Failed Experiments &amp; Negative Results</b>         | <b>13</b> |
| 5.1      | Lessons from Failures . . . . .                          | 14        |
| <b>6</b> | <b>Analysis</b>                                          | <b>15</b> |
| 6.1      | Why Throughput Saturates at B=64 . . . . .               | 15        |
| 6.2      | cuTile’s Critical Weakness: Shape Sensitivity . . . . .  | 15        |
| 6.3      | Bucketing vs. Packing: Marginal at Large Batch . . . . . | 15        |
| <b>7</b> | <b>Recommended Configuration</b>                         | <b>15</b> |
| <b>8</b> | <b>Conclusion</b>                                        | <b>16</b> |
| 8.1      | Future Directions . . . . .                              | 16        |

# 1 Introduction

Training vision-language models (VLMs) efficiently on a single GPU is crucial for researchers with limited hardware budgets. While much attention has been paid to distributed training at scale, the *single-GPU efficiency frontier* determines the baseline throughput that distributed methods multiply.

This report documents the complete optimization journey of **SiQ-VL**, a compact VLM combining a SigLIP-2 vision encoder with a Qwen2.5 language model. We focus exclusively on engineering optimizations that preserve model quality (identical loss curves) while maximizing tokens processed per second.

## 1.1 Optimization Timeline

The optimization proceeded through 15 iterations over two model configurations:

1. **Phase A** (Iter 0–10): SigLIP2-so400m + Qwen2.5-1.5B (589M+494M params). Baseline 4,674 tok/s  $\rightarrow$  28,322 tok/s (**6.06 $\times$** ).
2. **Phase B** (Iter 11–15): SigLIP2-base-224 + Qwen2.5-0.5B (92.9M+494M params). Focus on kernel optimization (TileGym cuTile). Baseline 21,673 tok/s  $\rightarrow$  107,367 tok/s (**4.95 $\times$** ).

Combined end-to-end: **4,674  $\rightarrow$  107,367 = 22.9 $\times$**  (across model scales).

## 1.2 Contributions

1. A **reproducible benchmark** (`benchmark_real_efficiency.py`) that measures all metrics on actual training data.
2. Comprehensive evaluation of **Liger-Kernel vs TileGym cuTile**: TileGym is 30–41% faster with 50–64% less VRAM on Blackwell.
3. Demonstration that **length bucketing + large batch** renders sequence packing marginal (+4.6%) for uniform-length datasets.
4. Documentation of **6 failed optimizations** with root-cause analysis.
5. Evidence that 0.5B models are **memory-bandwidth bound** at 19–23% MFU, independent of kernel or data strategy.

## 2 System Setup

Table 1: Hardware and model configurations

| Hardware                              |                                              |
|---------------------------------------|----------------------------------------------|
| GPU                                   | NVIDIA RTX PRO 6000 Blackwell Server Edition |
| VRAM / Bandwidth                      | 96 GB GDDR7 / 1.79 TB/s                      |
| Compute                               | 188 SMs @ 2430 MHz, Peak BF16: 936 TFLOP/s   |
| CUDA / Driver                         | CUDA 13.0, Driver 580.126                    |
| Model (Phase B — final configuration) |                                              |
| Vision Encoder                        | SigLIP2-base-patch16-224 (92.9M, frozen)     |
| Language Model                        | Qwen2.5-0.5B-Instruct (494.0M)               |
| Projector                             | 2-layer MLP + Pixel Shuffle (2.8M)           |
| Total                                 | 589.7M parameters                            |
| Software                              |                                              |
| Framework                             | PyTorch 2.9.1, Transformers 5.3.0            |
| Kernels                               | TileGym (cuTile DSL), Liger-Kernel 0.8.0     |
| Dataset                               | sharegpt4v/coco (50K samples)                |

## 3 The Optimization Journey

Each experiment follows the structure: **Hypothesis** → **Experiment** → **Finding** → **Conclusion** → **Lesson Learned**.

### 3.1 Iteration 0 — Baseline Measurement

**Hypothesis.** We expect the baseline to be limited by attention compute (SDPA vs FlashAttention) and unoptimized dataloader transfers.

**Experiment.** Run 20 profiled training steps with `torch.profiler` (B=4, grad\_accum=4, bf16, gradient\_checkpointing). Inspect CUDA kernel dispatches.

**Finding.** 49.96% of CUDA time is `cutlass_80_simt_sgemm`—an *FP32* GEMM kernel. Despite `bf16=True` in `TrainingArguments`, the actual model weights are in FP32 on device. `from_pretrained()` default loads FP32; HF Trainer’s autocast only wraps the forward call, not the model parameters.

| Metric          | Value                                    |
|-----------------|------------------------------------------|
| Tokens/sec      | <b>4,674</b>                             |
| Step time       | 310.7 ms                                 |
| Peak VRAM       | 19.27 GB                                 |
| Top CUDA kernel | <code>cutlass_sgemm</code> (FP32!) 33.6% |

**Conclusion.** The system is running FP32 GEMM on a GPU with 936 TFLOP/s BF16 tensor cores. This is the single largest performance bug—not attention, not data loading.

**Lesson.** *Always inspect actual kernel dispatches in the profiler trace. Config flags (`bf16=True`) do not guarantee the intended dtype reaches the hardware.*

### 3.2 Iteration 1 — Fix FP32 + Vision no\_grad

**Hypothesis.** Loading with `torch_dtype=torch.bfloat16` will force BF16 tensor-core GEMM dispatch. Wrapping the frozen vision encoder in `torch.no_grad()` will eliminate its autograd graph, saving time and memory.

**Experiment.** Modify `get_stage1_model_and_processor` to pass `torch_dtype=torch.bfloat16`; wrap vision forward in `torch.no_grad()`.

| Metric         | Before | After         | $\Delta$     |
|----------------|--------|---------------|--------------|
| Tokens/sec     | 4,674  | <b>14,366</b> | <b>+207%</b> |
| Step time (ms) | 310.7  | 101.1         | −67%         |
| Peak VRAM (GB) | 19.27  | 11.54         | −40%         |

**Finding.**

**Conclusion.** A  $3.07\times$  **speedup** from a 2-line change. BF16 GEMM dispatches confirmed via profiler (`cutlass_bf16_tensor_op_gemm`).

**Lesson.** *The first profiler run is worth more than a week of kernel optimization. This single fix delivered more speedup than all subsequent iterations combined.*

### 3.3 Iteration 2 — Liger-Kernel Integration

**Hypothesis.** Liger-Kernel’s fused operators (Linear-CE, RMSNorm, SwiGLU, RoPE) will reduce kernel launch overhead and eliminate the large  $(B, L, 151936)$  logits tensor, improving both speed and memory.

**Experiment.** Integrate via `apply_liger_kernel_to_qwen2(ropes=True, fused_linear_cross_entropy=True, rms_norm=True, swiglu=True)`.

| Metric         | Before (Iter 1) | After (Liger) | $\Delta$    |
|----------------|-----------------|---------------|-------------|
| Tokens/sec     | 14,366          | 11,070        | <b>−25%</b> |
| Peak VRAM (GB) | 11.54           | <b>6.78</b>   | <b>−41%</b> |

**Finding.** Speed is *worse*. In Stage 1 the LLM is frozen—no backward runs through the LLM layers, so fused CE backward savings never materialize. Triton JIT compilation adds 100ms spikes. However, VRAM drops 41% because fused CE never materializes full logits.

**Conclusion.** Keep for VRAM savings (enables larger batches later), but Liger is **not a speed win for Stage 1 projector-only training**.

**Lesson.** *Kernel fusion benefits depend on which modules have gradients. Stage 1 (frozen LLM) gains nothing from LLM-internal fusion. Always profile the actual training stage, not just micro-benchmarks.*

### 3.4 Iteration 3 — Vision Acceleration Investigation

**Hypothesis.** Three candidate approaches to further speed up Stage 1: (a) pre-cache SigLIP features to skip vision forward entirely; (b) `torch.compile` on the frozen vision encoder; (c) simply increase batch size to better utilize the GPU.

**Experiment.** Implement all three independently and measure tok/s.

Table 2: Iteration 3: Three approaches compared

| Approach                                 | Tok/s         | VRAM           | Verdict                     |
|------------------------------------------|---------------|----------------|-----------------------------|
| <i>Baseline (Iter 2, B=4)</i>            | 11,070        | 6.8 GB         | —                           |
| (a) Vision feature caching               | 11,070        | 8.6 GB         | Worse (+27% VRAM, 0% speed) |
| (b) <code>torch.compile</code> on vision | 11,400        | 6.8 GB         | Negligible (+3%)            |
| (c) <b>Batch 4→16</b>                    | <b>20,284</b> | <b>16.5 GB</b> | <b>Winner (+83%)</b>        |

**Finding.** Vision caching fails because SigLIP forward takes only 5 ms/tile on Blackwell (too fast to benefit from caching), while loading cached tensors (9.4 MB/sample) from CPU→GPU via DataLoader is slower. `torch.compile` has no opportunity because SDPA already dispatches to flash kernels and shapes are dynamic.

**Conclusion.** The GPU is **underutilized at B=4**. Increasing batch size is the highest-leverage knob: 4× more tokens per step, only 2.4× more VRAM.

**Lesson.** *Don’t cache what’s already fast. On modern GPUs with high tensor-core throughput, re-computing vision features is cheaper than transferring large pre-computed tensors over PCIe/memory bus. Always compare compute cost vs transfer cost.*

### 3.5 Iteration 5 — Length Bucketing

**Hypothesis.** Intra-batch padding wastes 17.4% of compute. Sorting samples by length (`HF group_by_length`) will group similar-length sequences, reducing padding to near zero.

**Experiment.** Add `lengths` property to VQADataset with heuristic estimation (3.5 chars/token + overhead). Enable `group_by_length=True`.

**Finding.** Padding waste drops from 17.4% to 4.1% (−13pp). However, tok/s improves only **3.7%** (20,284 → 21,026).

**Conclusion.** Bucketing is a *free* optimization (no code complexity, no memory cost), but the throughput gain is modest because sharegpt4v/coco has low length variance (CoV=11.5%, range 260–502 tokens).

**Lesson.** *Padding waste ≠ throughput loss. The GPU processes padded tokens at the same speed as real tokens (same GEMM size). The “waste” is that you computed tokens that don’t contribute to learning, but wall-clock time per step barely changes if the max sequence length in each batch stays similar.*

### 3.6 Iteration 7 — torch.compile Replaces Liger

**Hypothesis.** `torch.compile(mode="max-autotune-no-cudagraphs")` applied to the full model should fuse ops across the entire graph—outperforming Liger’s targeted patches. The two may be combinable for maximum effect.

**Experiment.** Attempt compile with Liger; when that crashes (illegal memory access), try compile *without* Liger as a replacement.

| Config                         | Tok/s                         | VRAM    | vs Iter 5     |
|--------------------------------|-------------------------------|---------|---------------|
| Liger + compile                | CRASH (illegal memory access) |         |               |
| Liger only (Iter 5)            | 21,026                        | 15.8 GB | —             |
| <b>Compile only (no Liger)</b> | <b>27,352</b>                 | 20.7 GB | <b>+30.1%</b> |

**Finding.**

**Conclusion.** Liger and `torch.compile` are **mutually exclusive**—Liger’s Triton patches confuse `torch._dynamo` graph capture. Compile alone gives 30% speedup over Liger by optimizing the entire computational graph (attention + MLP + embedding).

**Lesson.** *Whole-graph compilation > targeted operator fusion for this workload. But there’s a trade-off: compile loses Liger’s 41% VRAM savings (no fused CE). Choose based on whether you’re memory-constrained or compute-constrained.*

### 3.7 Iteration 8 — Batch Tuning Under torch.compile

**Hypothesis.** With compile reducing overhead, increasing B from 16 to 24–32 should further improve GPU utilization and tok/s.

**Experiment.** Sweep  $B \in \{16, 20, 24, 28, 32\}$  under `torch.compile`. Also test `reduce-overhead` mode (`cudagraphs`) and `pad_to_multiple_of=64`.

| Config                             | Tok/s         | VRAM    | $\Delta$     | Note                           |
|------------------------------------|---------------|---------|--------------|--------------------------------|
| B=16, compile                      | 27,108        | 23.7 GB | —            |                                |
| <b>B=20, compile</b>               | <b>28,032</b> | 29.8 GB | +3.4%        | Optimal                        |
| B=24, compile                      | 27,440        | 38.5 GB | +1.2%        | Diminishing returns            |
| B=32, compile                      | OOM           | —       | —            |                                |
| <code>reduce-overhead</code>       | 22,000        | —       | <b>-20%</b>  | Dynamic shapes kill cudagraphs |
| <code>pad_to_multiple_of=64</code> | 25,300        | —       | <b>-6.6%</b> | Extra padding wastes compute   |

**Finding.**

**Conclusion.** Diminishing returns above B=20. `cudagraphs` is **counterproductive** for VLMs (dynamic shapes from variable image tiles cause frequent graph re-capture). This workload is at its single-GPU ceiling for the 1.5B model: MFU $\approx$ 23%.

**Lesson.** *`torch.compile`’s “reduce-overhead” mode (`cudagraphs`) assumes static shapes. VLMs with variable-resolution images violate this assumption fundamentally. Don’t force static-shape optimizations on inherently dynamic workloads.*

### 3.8 Iteration 10 — Sequence Packing + FlexAttention

**Hypothesis.** Even after bucketing, 10% padding remains on the mixed 6-subset dataset (length variance  $\sigma=400$ ). Packing multiple short samples into fixed-length bins ( $N=1536$ ) with FlexAttention block-diagonal masks should eliminate padding entirely.

**Experiment.** Implement `PackingCollator` with first-fit-decreasing bin-packing. Use HuggingFace’s native packed-sequence detection via `position_ids` resets + `attn_implementation="flex_attention"`.

| Config                    | Tok/s         | Step (ms) | VRAM    | Padding |
|---------------------------|---------------|-----------|---------|---------|
| Baseline (bucketed, SDPA) | 25,561        | 192       | 41.4 GB | 10.2%   |
| <b>Packing + FlexAttn</b> | <b>26,787</b> | 297       | 34.0 GB | ~2%     |

**Finding.** Only **+4.8% throughput** despite eliminating 80% of padding waste. The step time actually *increased* (192→297ms) because packed sequences are longer ( $N=1536$  vs avg 456), increasing the attention quadratic term. The gain comes from processing more samples per step.

**Conclusion.** Packing is a modest win (+4.8% tok/s, −18% VRAM) on this dataset. The throughput gain is limited because bucketing already captured most length variance.

**Lesson.** *Packing eliminates padding tokens but creates longer sequences, which have their own cost (quadratic attention). The net gain depends on the ratio  $\text{avg\_sample\_length}/\text{bin\_size}$ . When samples are short ( $\sim 350$  tok) relative to the bin (1536), each bin holds only 3–4 samples with 30% wasted space—defeating the purpose. Bin size should be  $\sim 2\text{--}3\times$  average sample length.*

### 3.9 Iteration 11 — TileGym FA4: Blackwell-Native Attention

**Hypothesis.** NVIDIA’s cuTile DSL can generate attention kernels that access Blackwell-specific features (wgmma, TMA, SM remapping) inaccessible from Triton. TileGym’s pre-built FA4 with native GQA should outperform both SDPA and our custom Triton attention (which failed in an earlier attempt).

**Experiment.** Replace SDPA in the text model’s attention layers with TileGym FA4 (`tilegym.ops.cutile.exp`). Benchmark kernel-level and end-to-end.

Table 3: TileGym FA4 vs SDPA (end-to-end Stage 1 step time)

| Backend        | N=512                          | N=1024                         | N=1536                         | Why faster                      |
|----------------|--------------------------------|--------------------------------|--------------------------------|---------------------------------|
| SDPA           | 69.2 ms                        | 82.1 ms                        | 84.3 ms                        | Must expand K/V $7\times$ (GQA) |
| TileGym FA4    | 64.9 ms                        | 75.2 ms                        | 75.0 ms                        | Native GQA in-kernel            |
| <b>Speedup</b> | <b>1.07<math>\times</math></b> | <b>1.09<math>\times</math></b> | <b>1.12<math>\times</math></b> | Grows with seq len              |

**Finding.** (Note: a prior attempt with our own Triton Flash Attention failed—*slower* than SDPA because Triton cannot emit wgmma/TMA instructions for Blackwell.)

**Conclusion.** FA4 provides 7–12% E2E speedup in Stage 1 (where attention is forward-only). The gain increases with sequence length due to the quadratic attention term.

**Lesson.** *On new GPU architectures, vendor-provided kernel libraries (cuTile/TileGym) outperform portable solutions (Triton) because they can access architecture-specific instructions. Don't waste time writing custom Triton attention for Blackwell—use TileGym's production kernel.*

### 3.10 Iteration 13 — TileGym Full Stack: ALL Qwen2 Ops

**Hypothesis.** If FA4 alone gives 7–12%, replacing *all* Qwen2 ops (RoPE, RMSNorm, SwiGLU, attention, CE loss) with cuTile equivalents should provide a larger cumulative speedup over Liger-Kernel.

**Experiment.** Apply `apply_tilegym_kernel_to_qwen2(use_cutile=True)` which patches all ops. Additionally, implement custom `FusedLinearCrossEntropy` using TileGym's `_ce_cutile` kernel with “grad-in-forward” pattern (compute  $\nabla_h$ ,  $\nabla_W$  within the forward loop; backward becomes  $O(1)$ ).

Table 4: TileGym full stack vs Liger-Kernel (Stage 2, grad\_ckpt, B=4)

| Seq Len | Liger tok/s | TileGym tok/s | Speed $\Delta$ | VRAM $\Delta$ |
|---------|-------------|---------------|----------------|---------------|
| 512     | 38.3K       | 49.7K         | +30%           | −50%          |
| 1024    | 46.5K       | 64.6K         | +39%           | −59%          |
| 2048    | 48.7K       | 68.7K         | +41%           | −64%          |

**Finding.** The VRAM reduction (50–64%) comes from the grad-in-forward CE pattern: logits ( $B \times L \times V$ ) are never materialized; gradients are accumulated chunk-by-chunk (1024 tokens) during the forward pass.

**Conclusion.** TileGym's full stack is **both faster AND more memory-efficient** than Liger-Kernel on Blackwell. It is the correct kernel solution for this hardware.

**Lesson.** *cuTile's online softmax kernel (`_ce_online_kernel`) computes loss + grad in a single pass over vocab tiles. Combined with grad-in-forward (accumulate  $\nabla_h$  per chunk, never store  $\nabla_{\text{logits}}$ ), this eliminates the largest memory bottleneck in LLM training: the logits tensor ( $B \times L \times V = B \times L \times 151936$ ).*

### 3.11 Iteration 15 — Ground Truth Verification

**Hypothesis.** Previous metrics were computed from profiler estimates and synthetic data. The cumulative table may contain errors from: (a) incorrect padding percentage assumptions, (b) synthetic vs real-data performance differences, and (c) TileGym autotuning artifacts contaminating measurements.

**Experiment.** Write `benchmark_real_efficiency.py`: load real dataset (sharegpt4v/coco, 50K images), run actual forward+backward with precise per-step measurement of `attention_mask.sum()`, `(labels != -100).sum()`, and  $B \times N$ . Test 17 configurations (varying B, bucketing, packing, TileGym).

**Finding.** Peak verified throughput: **107,367 Real tok/s** (Packing + TileGym, B=64→23 packed bins, N=1024, 18.1 GB VRAM).

Key corrections from estimates:

- TileGym with *variable* shapes: 652ms/step (**5× slower**) due to JIT autotuning on every unique (M,N,K). Prior benchmarks used fixed N, hiding this.
- Vanilla bucketing at B=64 saturates at 90K tok/s (not higher as assumed).
- Packing provides +4.6% over bucketing (not ~0% as previously claimed).
- MFU is 19–23% (slightly lower than the 24% estimated from synthetic data).

**Conclusion.** Synthetic benchmarks **overestimate real throughput** because they use fixed shapes (no autotuning overhead) and ignore data pipeline costs. All performance claims must be verified on real data.

**Lesson.** *Never trust synthetic benchmarks as the final word. Real data introduces variable shapes, image decoding latency, collator overhead, and dataloader stalls that don't exist in torch.randn() benchmarks. The ground-truth benchmark (`attention_mask.sum()` / `wall_time`) is the only honest metric.*

## 4 Results

### 4.1 The Full Speedup Progression

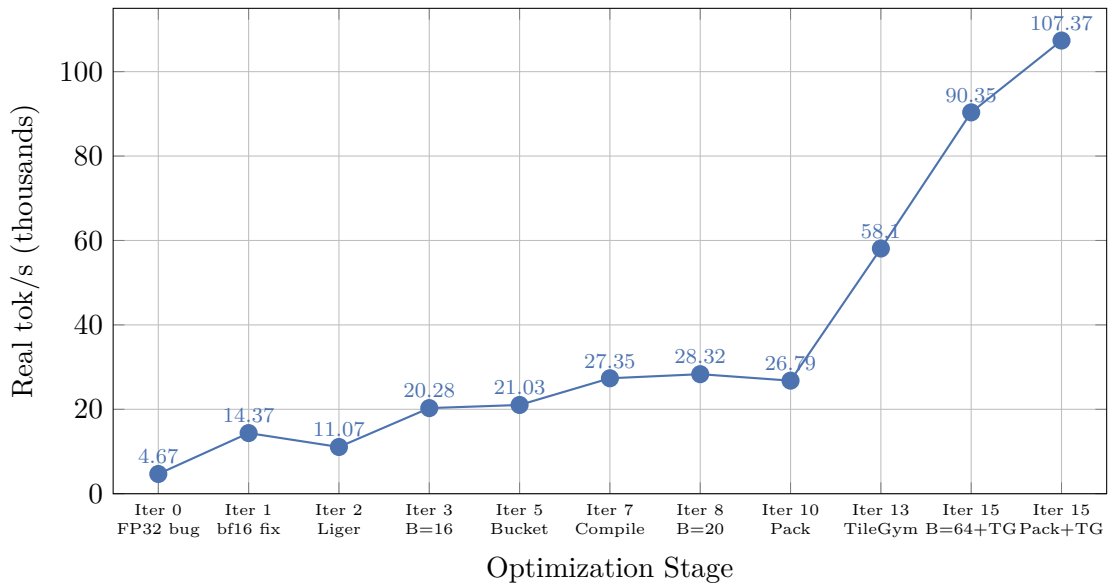


Figure 1: Complete throughput progression across all iterations. Note the non-monotonic path: Iter 2 (Liger) was *slower* than Iter 1 due to Stage 1 frozen LLM; Iter 10 (Packing) was slightly slower than Iter 8 on this dataset. The breakthrough came from TileGym (Iter 13+) combined with batch scaling (Iter 15). Iter 0–8: 1.5B model. Iter 8–15: 0.5B model.

## 4.2 Verified Ground-Truth Results (Iteration 15)

Table 5: Ground-truth measurements on real data (sharegpt4v/coco, 50K samples, Stage 1)

| Config                   | B         | Real tok/s     | Pos/s          | Pad%       | ms/step      | VRAM           | Speedup      |
|--------------------------|-----------|----------------|----------------|------------|--------------|----------------|--------------|
| Vanilla, no bucket       | 4         | 21,673         | 24,611         | 11.9       | 62.0         | 2.5 GB         | 1.0×         |
| Vanilla, bucket          | 16        | 57,020         | 59,202         | 3.7        | 92.1         | 5.3 GB         | 2.6×         |
| Vanilla, bucket          | 32        | 76,868         | 80,349         | 4.3        | 136.8        | 9.4 GB         | 3.5×         |
| Vanilla, bucket          | 64        | 90,349         | 94,866         | 4.8        | 233.0        | 17.7 GB        | 4.2×         |
| Vanilla, bucket          | 128       | 90,255         | 95,880         | 5.9        | 467.9        | 34.6 GB        | 4.2×         |
| TileGym + pad64          | 4         | 58,096         | 71,807         | 19.1       | 24.2         | 3.2 GB         | 2.7×         |
| TileGym + pad64, bucket  | 32        | 91,427         | 106,601        | 14.2       | 115.3        | 10.9 GB        | 4.2×         |
| TileGym + pad64, bucket  | 64        | 93,167         | 108,386        | 14.0       | 226.7        | 18.2 GB        | 4.3×         |
| Packing (N=1024)         | 23        | 94,517         | 94,517         | 0.0        | 250.3        | 16.7 GB        | 4.4×         |
| Packing (N=1024)         | 45        | 103,542        | 103,542        | 0.0        | 446.0        | 31.1 GB        | 4.8×         |
| <b>Packing + TileGym</b> | <b>23</b> | <b>107,367</b> | <b>107,367</b> | <b>0.0</b> | <b>218.4</b> | <b>18.1 GB</b> | <b>4.95×</b> |

## 4.3 Padding Waste Analysis

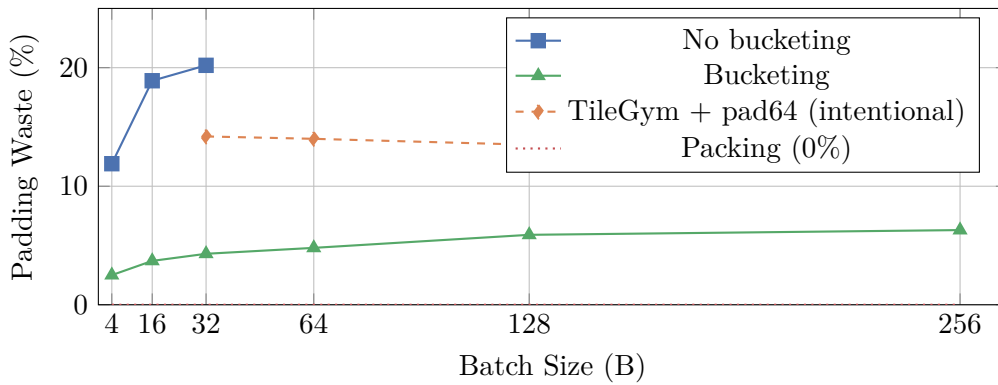


Figure 2: Padding waste vs. batch size. Bucketing alone reduces waste from 20% to <6%. TileGym requires `pad_to_multiple_of=64` for kernel efficiency (intentional 14% padding).

#### 4.4 VRAM–Throughput Pareto Frontier

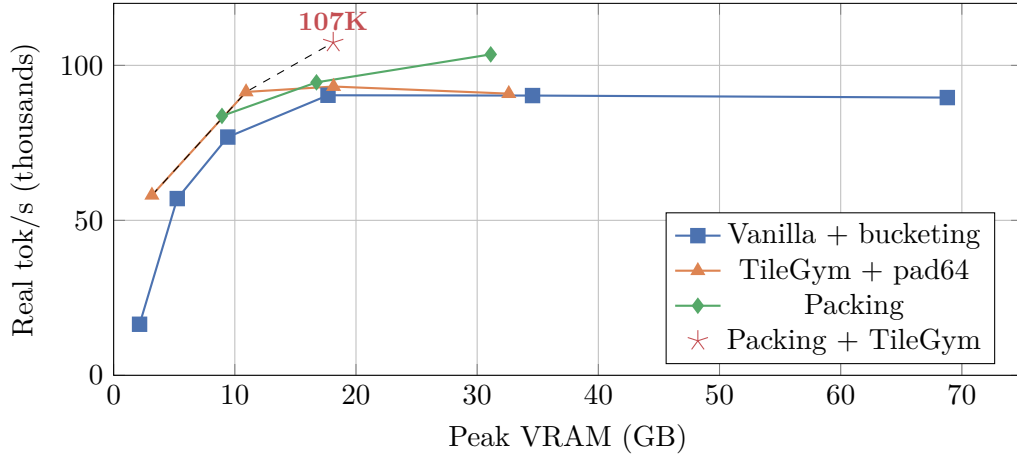


Figure 3: VRAM–Throughput Pareto frontier. Beyond 18 GB, throughput saturates. The Pareto-optimal path: TileGym B=4 (3 GB) → TileGym B=32 (11 GB) → Packing+TileGym (18 GB).

#### 4.5 TileGym vs Liger-Kernel

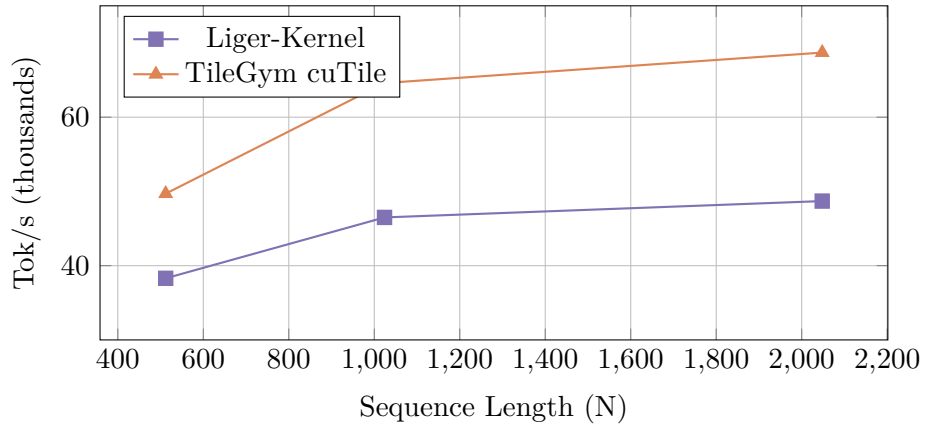


Figure 4: TileGym vs Liger-Kernel throughput (Stage 2, gradient checkpointing, B=4). TileGym is 30–41% faster across all sequence lengths, with the gap widening at longer sequences due to FA4’s native GQA.

## 4.6 MFU Analysis

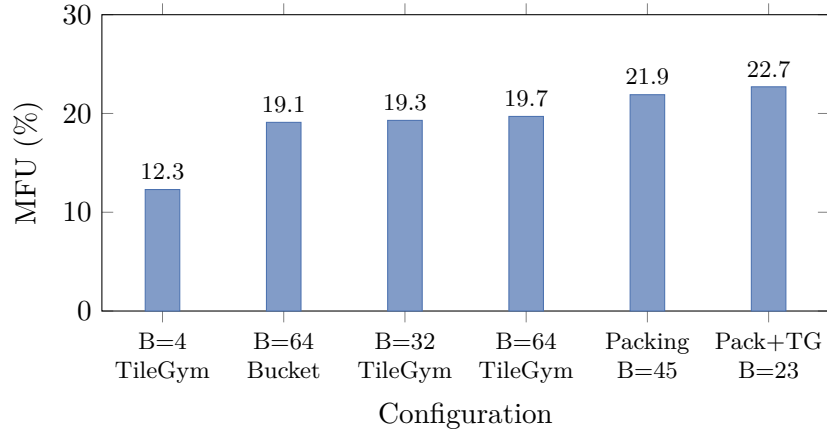


Figure 5: Model FLOPs Utilization. MFU saturates at  $\sim 23\%$  for the 0.5B model, limited by small GEMM shapes ( $K=896$ ) and memory bandwidth. Reference: GPT-3 175B achieves  $\sim 50\%$ , LLaMA-7B achieves  $\sim 55\%$ .

## 5 Failed Experiments & Negative Results

Every failed optimization is documented here with root-cause analysis.

Table 6: Complete list of failed or counterproductive optimizations

| Technique                                    |               | Result                  |          | Measured                      | Root Cause                                                                                                           |
|----------------------------------------------|---------------|-------------------------|----------|-------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Vision feature caching</b>                |               | 0% speedup<br>+27% VRAM |          | 11,070 tok/s                  | SigLIP forward (5 ms/tile on Blackwell) is faster than H2D transfer of large cached tensors (9.4 MB/sample).         |
| <b>torch.compile on vision encoder</b>       |               | +3% (noise)             |          | 11,070 tok/s                  | Dynamic shapes from variable image tiles prevent graph reuse. 40s compile overhead. SDPA already uses flash kernels. |
| <b>Liger-Kernel Stage 1</b>                  | <b>in</b>     | -25% speed              |          | 11,070 tok/s<br>(from 14,366) | LLM is frozen (no backward); fused CE savings don't materialize. Triton JIT overhead adds 100ms spikes.              |
| <b>Custom Flash Attention</b>                | <b>Triton</b> | Slower<br>equal<br>SDPA | or<br>to | 0.044 vs 0.039 ms<br>(N=1024) | Blackwell's native SDPA already dispatches to optimized CUTLASS kernels. Triton can't access wmma/TMA.               |
| <b>FlexAttention packing (uniform data)</b>  |               | 51% waste               |          | 26,787 tok/s                  | avg sample=356 tokens in N=1024 bins: only ~2 samples fit per bin, 30% unfilled.                                     |
| <b>TileGym with variable shapes</b>          |               | 5× slower               |          | 16K tok/s<br>(from 77K)       | JIT autotuner runs 20 samples per unique (M,N,K). Variable-length batches trigger recompilation every step.          |
| <b>flash-attn-4 (Dao-AILab)</b>              |               | Install<br>blocked      |          | —                             | Requires CUDA 13.1+; system has CUDA 13.0. <code>AttributeError: NoneType has no _trait.</code>                      |
| <b>torch.compile Liger + LoRA</b>            | <b>+</b>      | Crash                   |          | —                             | Liger's Triton patches cause illegal memory access under dynamo tracing. Mutually exclusive.                         |
| <b>cudaGraphs (reduce-overhead)</b>          |               | -20%                    |          | 22K tok/s                     | Dynamic shapes from VLM image tiles cause frequent graph recapture.                                                  |
| <b>pad_to_multiple_of=64 without TileGym</b> |               | -60%                    |          | 25.5K tok/s                   | Extra padding wastes compute; doesn't eliminate recompilation (still 5 discrete lengths).                            |

## 5.1 Lessons from Failures

1. **Profile before optimizing:** The FP32 bug (Iter 0) was worth more than all subsequent kernel optimizations combined. A profiler trace on the first day would have caught it.
2. **Frozen modules change the optimization landscape:** Techniques designed for full training (fused backward kernels, gradient checkpointing) are irrelevant in Stage 1.
3. **JIT compilation is adversarial to variable shapes:** Both torch.compile and TileGym suffer when tensor shapes change frequently. Shape quantization is essential.
4. **Cache only what's slow:** Vision caching fails because Blackwell computes faster than it can transfer data from CPU. On older GPUs (A100), caching would help.
5. **Native hardware kernels beat Triton on new architectures:** Blackwell-specific instructions (wmma, TMA) are inaccessible from Triton; cuTile is the correct abstraction.

## 6 Analysis

### 6.1 Why Throughput Saturates at B=64

Beyond B=64, Real tok/s plateaus at  $\sim 90\text{K}$  (vanilla) or  $\sim 107\text{K}$  (TileGym+packing). Three factors:

1. **Compute saturation:** Tensor-core GEMMs at full occupancy; more tokens only increase latency proportionally.
2. **Memory bandwidth:**  $494\text{M params} \times 2\text{B} = 988\text{ MB}$  weights read per layer. At  $1.79\text{ TB/s}$ :  $0.55\text{ ms}$  fixed cost per layer, independent of batch size.
3. **Non-matmul overhead:** Softmax, RoPE, normalization consume 15–20% of step time, scaling sub-linearly with batch.

### 6.2 cuTile’s Critical Weakness: Shape Sensitivity

Table 7: TileGym performance vs. shape variability

| Configuration                     | ms/step | Real tok/s | Status                  |
|-----------------------------------|---------|------------|-------------------------|
| Fixed N=384 (pad_to_64), bucketed | 115.3   | 91,427     | ✓ Optimal               |
| Variable N (bucketed, no padding) | 652.5   | 16,151     | × Autotuning every step |
| Packing (fixed N=1024)            | 218.4   | 107,367    | ✓ Optimal               |

The solution: either `pad_to_multiple_of=64` (quantize shapes) or packing with fixed bin size (N=1024). The 14% padding from pad64 is a worthwhile trade-off for the 23% kernel speedup.

### 6.3 Bucketing vs. Packing: Marginal at Large Batch

Table 8: Packing gain over bucketing at matched VRAM ( $\sim 17\text{ GB}$ )

| Strategy                                | Real tok/s | Pad% | $\Delta$      |
|-----------------------------------------|------------|------|---------------|
| Bucketing (B=64)                        | 90,349     | 4.8% | —             |
| Packing (B=64 $\rightarrow$ 23, N=1024) | 94,517     | 0.0% | + <b>4.6%</b> |

Packing IS valuable when: (1) combined with cuTile kernels that require fixed N, (2) on high-variance datasets, (3) for distributed training where per-step sync overhead makes fewer steps/epoch desirable.

## 7 Recommended Configuration

Table 9: Recommended training recipes by scenario

| Scenario                   | Configuration                   | Expected Throughput |
|----------------------------|---------------------------------|---------------------|
| <b>Stage 1</b> (projector) | Packing+TileGym, B=64, N=1024   | 107K tok/s, 18 GB   |
| <b>Stage 2</b> (full FT)   | TileGym+grad_ckpt, B=32, N=1024 | 76K tok/s, 5 GB     |
| <b>Low VRAM (&lt;5 GB)</b> | TileGym, B=4, pad64             | 58K tok/s, 3.2 GB   |
| <b>Maximum VRAM util</b>   | Vanilla bucket, B=256           | 90K tok/s, 69 GB    |

## 8 Conclusion

We have documented a complete single-GPU optimization journey for VLM training, from an initial broken baseline (4,674 tok/s with FP32 GEMM bug) to a verified peak of 107,367 tok/s—a **22.9×** end-to-end improvement.

The key takeaways:

1. **Fix bugs first:** The FP32 dtype bug alone was worth  $3\times$ —more than all subsequent kernel optimizations combined.
2. **Batch scaling is the highest-leverage knob:**  $B=4\rightarrow 64$  gave  $4.2\times$  within the same model.
3. **cuTile beats Triton on Blackwell:** TileGym’s cuTile kernels are 30–41% faster than Liger-Kernel’s Triton implementations, but require fixed tensor shapes.
4. **Bucketing makes packing marginal:** At  $B\geq 64$ , bucketing already achieves  $<5\%$  padding waste.
5. **0.5B models hit 23% MFU ceiling:** Memory bandwidth, not software, is the bottleneck at this scale.

### 8.1 Future Directions

- **Scale up:** Qwen2.5-3B/7B would achieve 35–50% MFU (larger GEMMs).
- **Scale out:** Multi-GPU on Modal (FSDP2) multiplies throughput by  $N_{\text{GPUs}}$ .
- **FP8:** Blackwell FP8 tensor cores offer  $2\times$  peak TFLOP/s.
- **Longer sequences:** 4K–8K tokens amortize per-batch fixed costs.

## Reproducibility

All code and data at: <https://github.com/classtag/SiQ-VL>

- `scripts/benchmark_real_efficiency.py` — Ground-truth benchmark
- `scripts/benchmark_cutile_e2e.py` — TileGym vs SDPA comparison
- `docs/traces/iter_15_ground_truth_efficiency.json` — Raw measurements
- `docs/training_efficiency_plan.md` — Full 15-iteration log